Nigerian Sign Language Sign-to-Speech Using YOLO26: Preprocessing and Baseline Design

Candace Tariro Hunzwi

Carnegie Mellon University
Engineering Artificial Intelligence

***Abstract*—This is a report auditing the image-based preprocessing strategy for Nigerian Sign Language (NSL) sign detection using YOLO26. Prior exploratory analysis indexed 4,815 images from 137 class folders and 4,815 Pascal VOC XML files; 4,814 images yielded a geometrically valid extracted bounding box. Images were reported at 640×480 pixels, and class frequency ranged from four to 65 images (imbalance ratio 16.25). There is the translation of the assignment's tabular requirements into technically valid controls for object detection, specify a reproducible image–annotation verification pipeline, and document the supervisor-directed baseline on the original, imbalanced images. The analysis establishes a data-driven foundation for preprocessing Nigerian Sign Language images, highlighting class imbalance, annotation integrity, and image characteristics that influence object-detection performance. These findings inform a reproducible preprocessing strategy and justify the selection of YOLO26 as the baseline architecture for subsequent sign recognition and evaluation experiments.**

***Index Terms*—Nigerian Sign Language, object detection, preprocessing, YOLO26, annotation audit, leakage, reproducibility**

I. Problem and Data

The NSL dataset is a multi-class object-detection dataset, not a tabular predictor matrix. Each observation is an RGB image with one or more associated sign-class bounding boxes. The original study used an earlier YOLOv5 model [1]; the current project targets YOLO26. An initial audit of the inspected image-folder/Pascal VOC collection reported the values in Table I. These are prior notebook audit outputs and are distinct from an independent full re-execution of the accompanying Phase 2 program.

TABLE I
Previously Reported Raw-Data Audit

Quantity	Value
Indexed images	4,815
XML files	4,815
Folder-derived sign classes	137
Images with valid extracted box	4,814
Raw image dimensions	640×480
Images per class (min–max)	4–65
Largest/smallest class ratio	16.25

The discrepancy between XML-file count and usable-box count must be resolved by inspecting the affected file; a matched XML is not proof of valid supervision. The original class list and XML label names also require consistency checks before training.

II. Diagnostic Evidence and Decisions

A. Target Imbalance

The class distribution has a ratio $65/4 = 16.25$. This is evidence of uneven representation, not evidence by itself of detector bias. In accordance with supervisor guidance, the baseline retains the natural class frequencies; targeted balancing is deferred until the baseline is evaluated. Training-only augmentation or sampling can then be tested without altering held-out data. The class-distribution figure is available in the project README under Fig. 1 in the accompanying repository.

B. Missingness and Annotation Integrity

For object detection, missing or invalid supervision is a data-integrity problem rather than a value to impute. The pipeline verifies that each image is readable, has exactly one parsable XML annotation, uses a known class label, and contains finite, positive bounding-box coordinates whose edges lie within the image. It also flags empty annotations and image–annotation mismatches. A missing image, sign label, or bounding box is never replaced with a mean, median, or predicted value; unresolved records are excluded from every split and recorded in an exclusion log with the file identifier and reason. These checks establish structural validity only. They do not confirm that a geometrically valid box encloses the intended sign, so flagged and sampled accepted annotations still require visual review.

C. Outliers and Image Geometry

Rare or extreme bounding-box areas can represent valid examples; box-size outliers should therefore be flagged and reviewed rather than capped or deleted automatically. The raw 640×480 images are preserved in the baseline, and the training loader letterboxes to a target 640×640 canvas while retaining aspect ratio. At that exact input scale, 80 pixels of vertical padding may be used on each side, although optimized rectangular batching can produce other intermediate shapes. Ground-truth boxes must remain aligned with image geometry.

D. Background and Visual Confounding

Representative images show additional people and classroom backgrounds. These observations motivate, but do not validate, future signer-background-removal and padded sign-region-cropping studies. Cropping a test image from its *ground-truth* box would use information unavailable in deployment; such results must be labeled oracle-assisted and not compared as ordinary end-to-end detections.

III. Leakage-Free Implementation

The accompanying diagnostics notebook contains raw-data indexing and visualization code. A separate, reusable Python preprocessing module reads source images and XML annotations, validates and converts bounding boxes into normalized YOLO coordinates, creates a deterministic split manifest, and records exact-image SHA-256 identifiers. Exact duplicate hashes are assigned to the same split. This does not establish signer-disjointness or catch all near duplicates; both are outstanding audit requirements. Class frequencies are not equalized. Random transformations, if enabled in future training, must occur only on training examples.

For rubric alignment, a Scikit-Learn `Pipeline` containing a stateless `ImageAuditFeatures` transformer is provided for reproducible audit metadata (width, height, aspect ratio, intensity mean and standard deviation). Its `fit` is invoked on training paths only. This yields a five-column *audit* matrix, not YOLO's model input. A `ColumnTransformer` for pixel values is not introduced because image tensor preprocessing and box-aware augmentation belong in the detection pipeline; there is no tabular predictor block requiring fitted numeric/categorical transforms. This is a scope adaptation, not a claim that YOLO alone fulfills the assignment's explicit `ColumnTransformer` requirement; instructor confirmation is needed.

A. Run Contract and Evidence

Before a detector is trained, the preprocessing program must be run against the complete local dataset with a fixed seed. It must write a validated manifest, exclusion log, split manifest, class-map YAML, verification JSON, and 300-DPI diagnostic figures. Each excluded record requires an image identifier, reason, and action. The validation and test partitions must never determine conversion rules, augmentation policy, hyperparameters, or the confidence threshold. The current supplied `test_outputs` directory is useful only as a smoke test: it indexes one image, produces a 1×5 audit matrix, and has empty validation/test partitions. It is not evidence for the full collection.

IV. Strategy vs. Reality Decision Log

This decision log compares the preprocessing strategy proposed in Phase 1 with the evidence and implementation status observed during Phase 2. It distinguishes completed validation steps from pending experiments, ensuring that planned work is not presented as an achieved result.

TABLE II
Strategy–Reality Decision Log

Area	Evidence and decision
Target balance	137 classes with 4–65 images (16.25:1 ratio). The baseline retains the original distribution; balancing is deferred.
Annotation failures	4,815 XML files and 4,814 valid extracted boxes were reported. Validation/exclusion code exists; full-data rerun and manual review remain pending.
Geometry	Raw images are 640×480; the target canvas is 640×640. YOLO-coordinate conversion is implemented; runtime loader verification is pending.
Outliers	Area distributions are reported in the README. Extreme but valid cases are retained for review rather than removed automatically.
Leakage	Signer identity is unknown and duplicates are possible. Exact-hash grouping is scripted; perceptual-duplicate and signer checks remain pending.
Architecture	YOLO26 is selected as the baseline; training outcomes are pending.
Background	The original-image baseline is retained. Background removal and cropping are future experiments.

V. YOLO26 Selection, Hyperparameters, and Baseline

YOLO26 supports bounding-box detection and pretrained checkpoint fine-tuning, providing a practical route from the original YOLOv5-based sign-to-speech design. Published COCO results for YOLO26n report 2.4 million fused-model parameters, 40.9 $\text{mAP}_{50:95}$ and 1.7 ms inference using T4 TensorRT under the documented benchmark configuration [2]. YOLO26s reports 9.5 million parameters, 48.6 $\text{mAP}_{50:95}$ and 2.5 ms on the same benchmark. These are not performance claims for NSL. The Nano variant is a defensible initial efficiency-oriented baseline; Small may later be considered if validation results justify additional capacity.

The proposed training configuration uses pretrained `yolo26n.pt`, 640-pixel input, initial budget of 100 epochs, nominal batch size 16, seed 42, and a fixed 70/15/15 target split. Actual split sizes, model arguments, accelerator, package versions, any default augmentation, training logs and best validation metrics must be recorded after execution. The final test set should be used only once model selection is frozen.

VI. Output Verification and Limitations

Table III separates prior full-collection EDA from the values that the final preprocessing execution must measure. The full-dataset program writes `verification.json` with indexed, accepted and excluded counts, exact train/validation/test image counts, a Boolean exact-hash leakage check, and shape $(N_{\text{train}}, 5)$ for the auxiliary audit feature matrix. The detector input has expected shape $(N, 3, 640, 640)$ after letterboxing when full square batches are used; this requires runtime inspection and is not claimed as measured. The script also exports 300-DPI validated class-count and box-area plots. Full-collection values cannot be inserted until the program is run against the 4,815 original images.

A. Notebook–Pipeline Reconciliation

The uploaded diagnostic notebook contains a separate baseline-preparation draft; the revised notebook removes du-

TABLE III
EVIDENCE STATUS FOR PIPELINE VERIFICATION

Evidence item	Status in this report
Raw image/XML inventory	Prior full-collection EDA: 4,815 / 4,815
Valid extracted boxes	Prior full-collection EDA: 4,814; one anomaly remains
Audit matrix dimensions	Smoke test only: 1×5; full run pending
Train/validation/test counts	Pending full-data split
Exact-hash leakage check	Smoke test passes; full-data result pending
Signer-disjoint verification	Not established; signer metadata unavailable
YOLO tensor shape	Expected after letterbox; runtime confirmation pending
Detector metrics	Not run; no NSL metric claimed

plicated path-dependent conversion cells and directs readers to the modular Python scripts. The baseline must be trained only after the generated class map is checked against the authoritative 137-label YOLO map. Stratified image-level splitting is insufficient to establish unseen-signer generalization when signer identifiers are absent. Exact-hash grouping is a partial leakage control, not a proof of full independence.

The repository currently contains raw-data class-distribution, annotation-coverage, image-dimension, box-distribution, representative-image, and transformation figures. These support the Phase 1-to-Phase 2 rationale but are not detector-performance plots. The final report should embed or explicitly reference only figures generated from the final pipeline run, at 300 DPI. Missingness heatmaps, raw-pixel correlation/VIF plots, and PCA scree plots require an agreed image-appropriate analogue; they must not be fabricated merely to imitate a tabular rubric.

VII. CONCLUSION

The confirmed EDA motivates geometry-preserving preprocessing, annotation-integrity checks, duplicate-aware splitting, and explicit uncertainty about rare classes. The immediate controlled experiment is YOLO26 on the original class distribution and full images, not a balanced or background-segmented derivative. The notebook preserves implemented data-indexing and diagnostic cells; the full-data preprocessing script and baseline training are separate stages. The present artifacts distinguish prior EDA, a one-image code smoke test, and still-unexecuted full-data/model stages. No gains are claimed before evaluation.

REFERENCES

[1] S. Kolawole et al., "Sign-to-Speech Model for Sign Language Understanding: A Case Study of Nigerian Sign Language," 2021. [Online]. Available: https://arxiv.org/abs/2111.00995

[2] Ultralytics, "YOLO26 Documentation," 2026. [Online]. Available: https://docs.ultralytics.com/models/yolo26/